Infosys Springboard Virtual Internship 7.0
Completion Report

Agentic AI for Smart Event Management

Batch Number : 3

Start date : 17th Aug 26

Team Name : Team C

Team Members : Swati Swagatika
Pushpa Uppu
Sravanthi Devarapalli
Yugant Sahu
Nithin Mamidi
Sonali Kapure
Susmitha
Sumit G

Internship Duration: 8 Weeks

.

1. Project Title

Agentic AI for Smart Event Management is an intelligent event-management solution that combines an agentic AI assistant with event discovery, registration, management, policy retrieval, and backend services.

2. Project Objective

The primary objective of the project is to develop an intelligent Smart Event Management platform powered by Agentic AI. The system is designed to understand natural-language user requests, determine the appropriate action, use relevant business tools or knowledge retrieval, and generate a useful response.

The project aims to simplify event discovery and management by bringing together user authentication, event information, registrations, venue-related information, an AI assistant, and a Retrieval-Augmented Generation (RAG) pipeline. The solution demonstrates how agentic workflows can connect business operations with organization-specific knowledge.

- Provide a centralized platform for event discovery and registration.
- Support event and registration management through a web-based application.
- Provide an AI assistant that can understand event-related questions and requests.
- Use an agentic workflow to select appropriate tools and actions.
- Use RAG, embeddings, and PGVector to retrieve relevant policy and FAQ information.
- Demonstrate practical integration of frontend, backend, database, and AI components.

3. Project Description in Detail

3.1 Overview

Agentic AI for Smart Event Management is a web-based event-management application with an AI-driven interaction layer. Users can log in, explore available events, view event details, register for events, and view their registrations. Administrators can manage event information and use the AI Assistant for event-related queries.

3.2 Core System Modules

- Authentication Module – supports user login and account management.
- Event Management Module – supports creation, viewing, updating, and management of event information.
- Event Discovery Module – allows users to browse available events and view event details.
- Registration Module – allows participants to register for events and view registration status.
- Venue Module – stores and provides venue-related information used by event workflows.
- Admin Dashboard – provides administrative views for event and platform management.
- AI Assistant – provides a natural-language interface for event-related questions and actions.
- Agent Module – uses an agentic workflow to interpret requests and select appropriate tools.
- RAG Module – retrieves relevant information from approved knowledge documents using embeddings and vector search.

3.3 Agentic AI Workflow

The agent acts as the decision-making layer between the user's request and the available system capabilities. A natural-language request is first interpreted. The agent then determines whether the request requires a business operation, knowledge retrieval, or a combination of both. The selected tool is executed, its result is observed by the agent, and the final response is generated for the user.

Stage	Description
1. User Request	User submits a natural-language question or action request.
2. Request Understanding	Agent identifies the user's goal and relevant intent.

3. Tool Selection	Agent selects a business tool or RAG-based knowledge retrieval.
4. Tool Execution	The selected service retrieves or performs the required operation.
5. Result Observation	Agent receives and interprets the tool result.
6. Final Response	Agent generates a clear response for the user.

3.4 RAG, Embeddings and PGVector

The RAG component enables the AI assistant to answer questions using project-specific knowledge rather than relying only on the language model's general knowledge. Knowledge documents such as event, registration, cancellation, venue, attendance, and FAQ policies can be processed and stored for semantic retrieval.

1. Load the knowledge documents.
2. Split documents into meaningful text chunks.
3. Generate vector embeddings for the chunks.
4. Store embeddings and metadata in PostgreSQL with PGVector.
5. Convert a user's knowledge-related query into an embedding.
6. Perform similarity search to retrieve relevant chunks.
7. Provide the retrieved context to the AI workflow.
8. Generate the final grounded response.

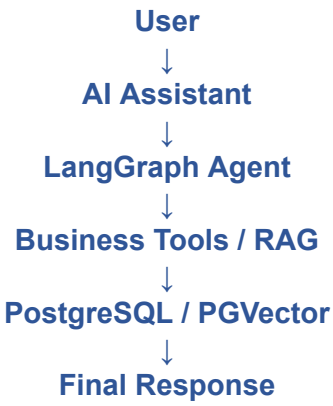
3.5 Technology Stack

Component	Technology / Tool
Frontend	React / Vite
Backend	Python / FastAPI
Database	PostgreSQL
Vector Database	PGVector
Agent Framework	LangGraph
RAG	Document loading, chunking, embeddings and similarity search
LLM Integration	OpenRouter-compatible LLM configuration
Version Control	Git / GitHub
Development	VS Code, Python virtual environment, browser-based testing

3.6 High-Level System Architecture

The project follows a layered architecture in which the frontend communicates with the FastAPI backend. The backend provides business services and connects the agentic AI layer with PostgreSQL and the PGVector-based retrieval layer.

Layer	Main Responsibility
User Interface	Login, event browsing, registration, admin dashboard and AI Assistant UI
API Layer	FastAPI endpoints for authentication, events, registrations and AI interaction
Agent Layer	LangGraph-based request understanding, routing, tool selection and response generation
Business Tools	Event search, event operations, registration and other application services
RAG Layer	Document processing, embeddings, PGVector retrieval and context preparation
Data Layer	PostgreSQL application data and vector representations



3.7 Real-World Relevance

The solution is relevant to organizations that conduct workshops, conferences, technical events, cultural programs, training sessions, and other activities. An agentic interface can reduce the need for users and administrators to navigate multiple screens for simple information requests. RAG additionally allows the assistant to answer organization-specific policy questions using controlled knowledge sources.

4. Timeline Overview

Week	Activities Planned	Activities Completed
Week 1	Project setup, orientation, Python and backend foundations	Project structure, environment setup and backend foundation established.
Week 2	Database and authentication	Database planning and authentication-related modules developed/integrated.
Week 3	Event APIs and event management	Event services and APIs were developed and connected with the application.

Week 4	LLM and AI Assistant	AI Assistant flow and LLM integration were developed and tested.
Week 5	LangGraph and agent tools	Agent workflow and tool-routing concepts were implemented/integrated.
Week 6	Embeddings, vector DB and RAG	Document processing, embeddings, PGVector and retrieval workflow were developed.
Week 7	Frontend, dashboard and integration	Frontend, backend, AI and database components were integrated and tested.
Week 8	Testing, logging, deployment and presentation	End-to-end testing, debugging, documentation and presentation preparation completed.

5a. Key Milestones

Milestone	Description	Date Achieved
Project Kickoff	Project orientation, requirements and scope definition	31 Aug 2026
Prototype / First Draft	Initial frontend/backend and core application flow	15 Sept 2026
Mid-Term Review	Core event-management and AI workflow demonstrated	01 Oct 2026
Final Submission	Integrated system, testing and final documentation	12 Oct 2026
Presentation	Final project demonstration and presentation	22 Oct 2026

5b. Project Execution Details

Frontend

The frontend provides the user-facing event-management experience, including authentication, event browsing, event details, registration, My Events, profile-related views, and the administrative interface. The AI Assistant provides a natural-language interaction point for supported event queries.

Backend

The backend is implemented using FastAPI and provides APIs for authentication, events, venues, registrations, and AI interaction. Services and routers separate application responsibilities and allow the frontend and AI workflow to communicate with the underlying data layer.

Database

PostgreSQL stores core application information such as users, events, venues and registrations. PGVector is used for vector representations required by semantic retrieval in the RAG pipeline.

Agent and AI Assistant

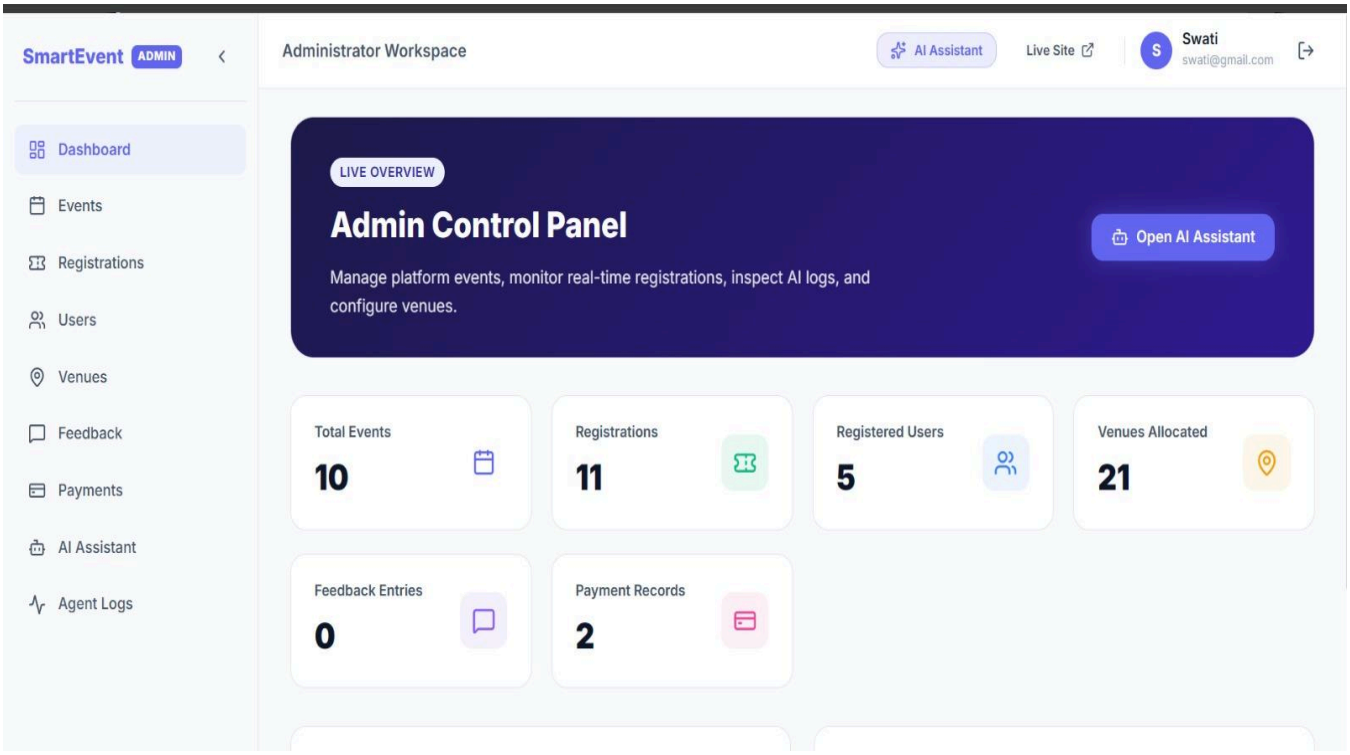
The AI Assistant sends user messages to the backend agent workflow. The agent interprets the request, selects the appropriate capability, executes the relevant tool or retrieval process, and returns a natural-language response.

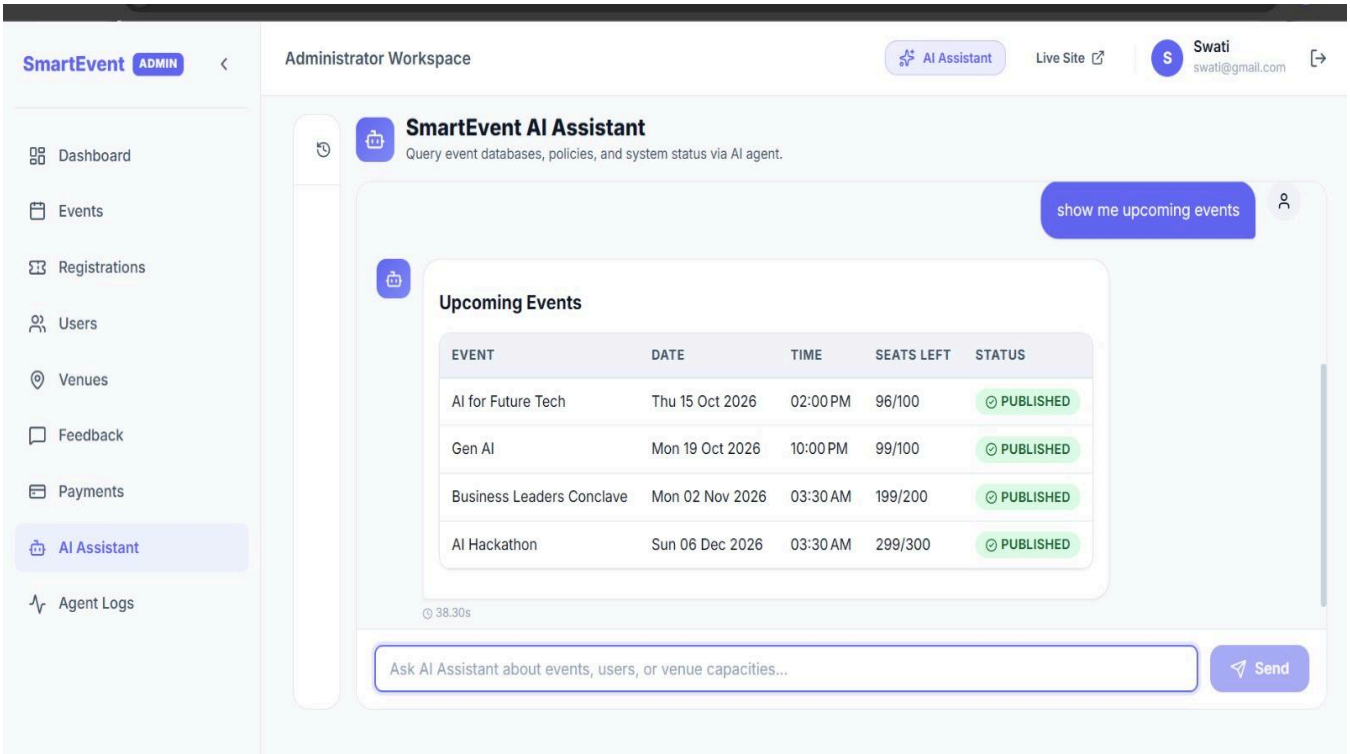
RAG Implementation

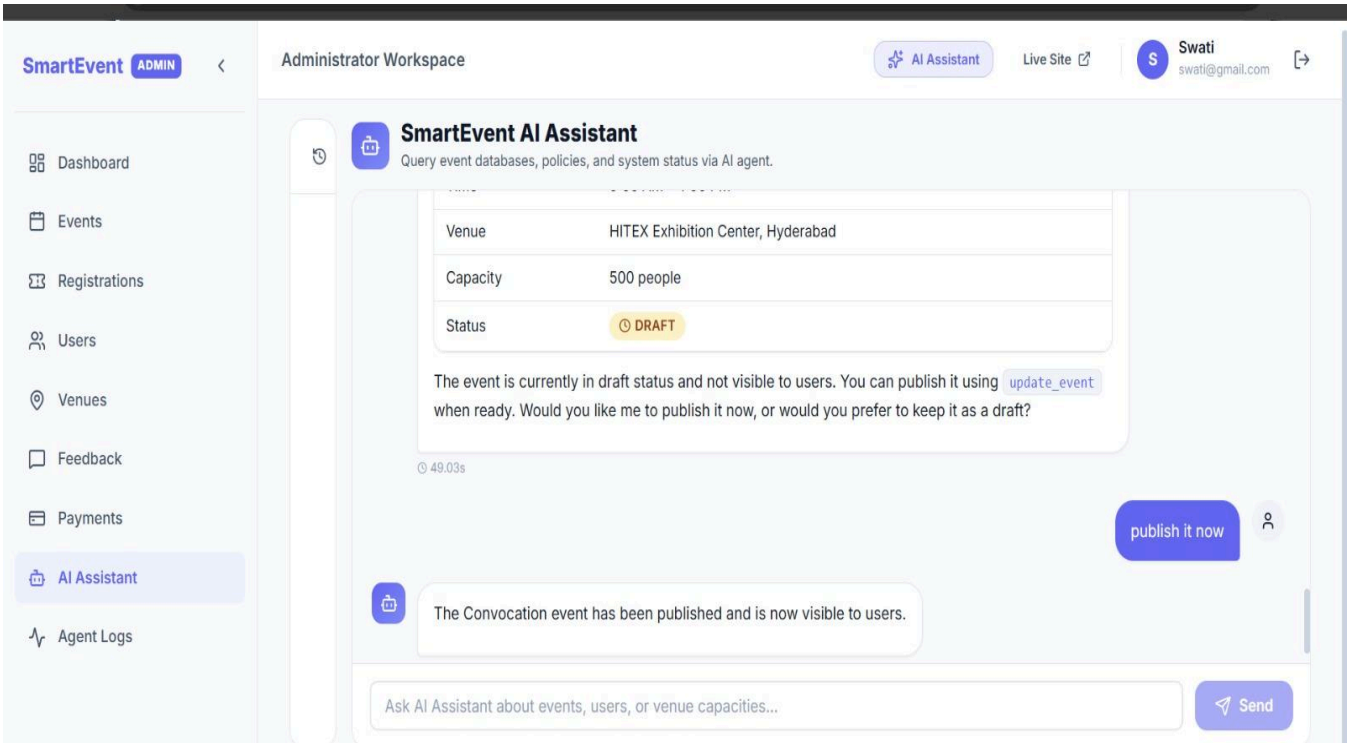
The RAG pipeline was designed around a sequence of document loading, chunking, embedding generation, vector storage, similarity search, and context injection. The retrieved information is then used by the AI workflow to produce a response grounded in the project's knowledge documents.

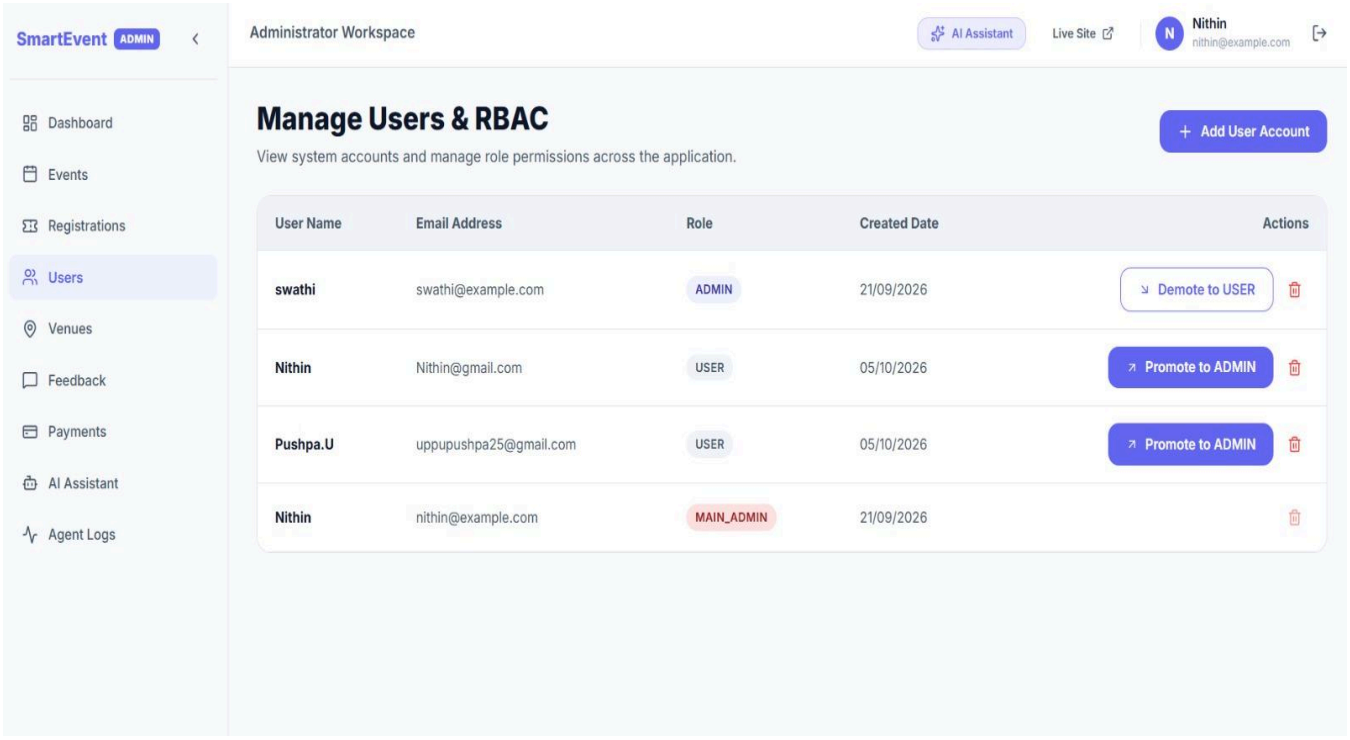
6. Snapshots / Screenshots

Insert actual screenshots from the final running application in the following places. The supplied sample reports use screenshots with a short description of the functionality shown.






SmartEvent ADMIN
Administrator Workspace
AI Assistant
Live Site
Swati
swati@gmail.com
Dashboard
Events
Registrations
Users
Venues
Feedback
Payments
AI Assistant
Agent Logs
SmartEvent AI Assistant
Query event databases, policies, and system status via AI agent.
Venue
HITEX Exhibition Center, Hyderabad
Capacity
500 people
Status
DRAFT
The event is currently in draft status and not visible to users. You can publish it using update_event when ready. Would you like me to publish it now, or would you prefer to keep it as a draft?
49.03s
publish it now
The Convocation event has been published and is now visible to users.
Ask AI Assistant about events, users, or venue capacities...
Send


SmartEvent ADMIN
Administrator Workspace
AI Assistant
Live Site
Nithin
nithin@example.com
Dashboard
Events
Registrations
Users
Venues
Feedback
Payments
AI Assistant
Agent Logs
Manage Users & RBAC
View system accounts and manage role permissions across the application.
Add User Account
User Name
Email Address
Role
Created Date
Actions
swathi
swathi@example.com
ADMIN
21/09/2026
Demote to USER
Nithin
Nithin@gmail.com
USER
05/10/2026
Promote to ADMIN
Pushpa.U
uppupushpa25@gmail.com
USER
05/10/2026
Promote to ADMIN
Nithin
nithin@example.com
MAIN_ADMIN
21/09/2026

7. Challenges Faced

Frontend–Backend Integration

Challenge: Different modules were developed by different team members. Connecting frontend screens, backend APIs, authentication, database services and AI functionality required repeated integration testing.

Resolution: The team tested each module independently and then verified the complete flow step by step.

AI Assistant Configuration

Challenge: The AI Assistant initially returned a generic error because the required LLM configuration was not available in the local environment.

Resolution: The configuration was checked, the missing API-key setup was identified, the environment was updated locally, and the backend was restarted and tested.

Agent Integration

Challenge: Understanding how the agent decides between business operations and knowledge retrieval required learning the relationship between the AI Assistant, LangGraph workflow and tools.

Resolution: The workflow was studied as separate stages: request understanding, tool selection, tool execution, result observation and final response.

RAG Integration

Challenge: Connecting document processing, embeddings, PGVector retrieval and the agent required careful understanding of how information moves through the pipeline.

Resolution: The RAG components were understood and tested step by step before integrating the retrieved context into the overall AI workflow.

Team Module Integration

Challenge: The project was divided among frontend, backend, database, authentication and AI contributors, so changes had to be integrated without disturbing other modules.

Resolution: Git branches, testing branches, controlled changes and repeated end-to-end testing were used to reduce integration issues.

8. Learnings & Skills Acquired

8.1 Technical Skills

- Understanding of Agentic AI concepts and agent-based application workflows.
- Hands-on understanding of LangGraph and tool-oriented agent design.
- RAG pipeline concepts including document loading, chunking, embeddings and similarity search.
- Practical understanding of PostgreSQL and PGVector for vector-based retrieval.
- FastAPI backend development and API integration.
- React/Vite frontend integration and browser-based testing.
- Git and GitHub workflow for collaborative project development.
- Environment configuration and debugging of AI-enabled applications.

8.2 Individual Contribution – RAG, Embeddings and PGVector

My primary contribution to the project was the RAG, embeddings and vector database component. I worked on understanding the complete retrieval pipeline: preparing knowledge documents, splitting content into chunks, generating embeddings, storing vectors in PGVector, performing similarity search, and making retrieved context available to the AI workflow.

This work helped me understand how an AI assistant can use project-specific knowledge instead of depending only on general model knowledge. I also gained practical experience in connecting my RAG component with the team's agent and backend integration.

8.3 Soft Skills

- Team communication and coordination.
- Problem-solving and debugging.
- Technical research and self-learning.
- Version-control discipline and collaborative development.
- Explaining technical work during project reviews and presentations.

9. Testimonials from Team

The project provided practical experience in collaborative software development and Agentic AI. Working across frontend, backend, database and AI modules helped the team understand the importance of clear interfaces between components.

- The project provided hands-on experience in developing an AI-enabled event-management application.
- Team collaboration helped in understanding how independently developed modules can be integrated into one system.
- Working with RAG, embeddings, PGVector and an agent workflow strengthened practical understanding of modern AI application development.
- Repeated testing and debugging improved confidence in identifying and resolving integration issues.

10. Conclusion

The Agentic AI for Smart Event Management project demonstrates how agent-based AI can be integrated with a practical event-management application. The system combines event and registration services with an AI Assistant that can understand natural-language requests and connect them with appropriate business capabilities or knowledge retrieval.

The project provided practical exposure to modern software and AI technologies including React, FastAPI, PostgreSQL, PGVector, embeddings, RAG, LangGraph and LLM integration. The internship also strengthened skills in teamwork, debugging, documentation and technical presentation.

Overall, the project helped bridge theoretical knowledge with practical implementation and provided valuable experience in building an integrated Agentic AI application.

11. Acknowledgements

I would like to express my sincere gratitude to Infosys Springboard for providing the Virtual Internship 7.0 opportunity and a valuable platform to gain practical experience in Agentic AI and software development.

I am thankful to our mentor, Mr. Thangarasu M, for his guidance and support throughout the project. His guidance helped us understand the project requirements and improve our knowledge of Agentic AI, backend development, Retrieval-Augmented Generation (RAG), and related technologies.

I would also like to thank all my Team C members — Swati Swagatika, Sravanthi Devarapalli, Yugant Sahu, Nithin Mamidi, Susmitha, Sumit G, and Sonali Kapure — for their collaboration, communication, and contributions across the frontend, backend, database, and AI modules.

Finally, I am grateful to everyone who contributed to this learning experience. Working on the Agentic AI for Smart Event Management project helped me strengthen my technical knowledge, problem-solving skills, and understanding of collaborative software development.